ST10462078

Tameez Lambat

SmartX Feedback

Smart-X — Part 2 Feedback Applied Report....1
1. Introduction....1
2. Predictive Alert and Recommendation System Improvements....2
2.1 Feedback Identified....2
2.2 Input Validation and Data Reliability....2
2.3 Improved Historical Reading Management....3
2.4 Improved Declining-Trend Detection....4
2.5 Context-Aware Recommended Actions....5
2.6 Duplicate Recommendation Prevention....6
2.7 Improved Actuator Command-Pattern Analysis....7
3. WPF Architecture Refactoring — Persistent Shell Navigation....8
3.1 Feedback Identified....8
3.2 Original Window-Based Navigation....8
3.3 Introduction of a Persistent Shell....9
3.4 ContentControl-Based Navigation....10
3.5 Converting Dashboard Windows into UserControls....11
3.6 Removing Independent Main Menu Navigation....11
4. Sensor Selection Persistence and Dashboard State Management....12
4.1 Feedback Identified....12
4.2 Remembering the Selected Sensor....13
4.3 Restoring the Selection....13
4.4 Preventing Unnecessary Dashboard Reinitialisation....14
4.5 Improving Input Preservation....15
5. Overall Impact of the Feedback Implementation....15
6. Conclusion....16

Smart-X — Part 2 Feedback Applied Report

1. Introduction

Following the feedback received during the development and evaluation of Part 2 of the Smart-X IoT Ecosystem Gateway, several improvements were implemented to address application functionality, usability, architectural consistency and the reliability of the predictive recommendation system.

The primary areas addressed were:

1. Improving the predictive alert and recommendation system.
2. Refactoring the WPF application architecture to use a persistent Shell-based navigation structure.
3. Improving sensor selection persistence and dashboard state management.
4. Improving the reliability of the telemetry dashboard and its interaction with the application.

The purpose of these changes was not simply to modify the appearance of the application, but to address underlying implementation issues and improve the overall user experience.

2. Predictive Alert and Recommendation System Improvements

2.1 Feedback Identified

The predictive recommendation system required improvements to make its output more useful, reliable and relevant to the behaviour of individual IoT devices.

A recommendation system should not only react to individual sensor readings. It should also be capable of analysing historical readings, identifying patterns and providing meaningful recommendations based on the observed behaviour of a device.

The improvements therefore focused on:

- Validating incoming sensor information before processing.
- Improving the detection of declining sensor readings.
- Reducing duplicate recommendations.
- Providing more meaningful recommended actions.
- Improving actuator command-pattern analysis.
- Ensuring that historical readings are evaluated chronologically.

2.2 Input Validation and Data Reliability

The `AnalyseEnvironmentalReading()` method was improved to validate incoming information before it enters the prediction history.

The implementation now checks:

- Whether the device MAC address is present.
- Whether the device identifier exceeds the permitted length.
- Whether the deployment location is provided and within the permitted length.
- Whether the sensor reading is a finite numeric value.
- Whether the timestamp is valid.

An example of the validation implementation is:

```
if (string.IsNullOrWhiteSpace(macAddress))
    throw new ArgumentException(
        "A device MAC address is required.",
        nameof(macAddress));

if (!float.IsFinite(sensorValue))
    throw new ArgumentOutOfRangeException(
        nameof(sensorValue),
        "The sensor reading must be a finite number.");

if (timestamp == DateTime.MinValue)
    throw new ArgumentException(
        "A valid reading timestamp is required.",
        nameof(timestamp));
```

This prevents invalid information from entering the prediction history and potentially influencing subsequent recommendations.

The device identifier and deployment location are also trimmed before being stored, helping maintain consistent values during processing.

Result: The recommendation system receives better-validated input, reducing the possibility of invalid sensor information affecting predictive analysis.

2.3 Improved Historical Reading Management

The system maintains a separate reading history for each device using a dictionary keyed by the device MAC address.

Each reading is stored with its timestamp and sensor value.

The implementation ensures that readings are maintained in chronological order:

```
readings.Add(new ReadingRecord
{
    Timestamp = timestamp,
    Value = sensorValue
});

readings.Sort(
    (a, b) => a.Timestamp.CompareTo(b.Timestamp));
```

The system also limits the amount of historical data retained for each device.

When the maximum reading count is exceeded, older records are removed so that the system retains the most recent readings.

This is important because predictive analysis depends on the sequence of readings over time. Processing readings in the correct chronological order helps ensure that trends are evaluated accurately.

Result: Each device has a managed history of sensor readings that can be used to identify patterns and changes in environmental conditions.

2.4 Improved Declining-Trend Detection

One of the most significant improvements was made to the declining-trend recommendation logic.

Previously, the recommendation logic primarily identified whether the latest three readings were consistently decreasing and whether the total decrease exceeded a fixed amount.

The improved implementation introduced more detailed analysis.

It now:

1. Retrieves the three most recent readings.
2. Ensures that their timestamps are distinct and increasing.
3. Confirms that the sensor values are consistently declining.
4. Calculates the total reduction in the sensor reading.
5. Calculates the elapsed time between the first and last readings.
6. Determines the rate of decline per minute.
7. Estimates how long it could take for the reading to reach the warning threshold.

The implementation first retrieves the latest three readings:

```
List<ReadingRecord> recentReadings = readings
    .OrderBy(r => r.Timestamp)
    .TakeLast(3)
    .ToList();
```

It then validates that the readings represent a genuine chronological decline:

```
if (first.Timestamp >= middle.Timestamp ||
    middle.Timestamp >= last.Timestamp)
    return;

if (first.Value <= middle.Value ||
    middle.Value <= last.Value)
    return;
```

The rate of decline is calculated using the difference between the first and last readings divided by the elapsed time:

```
double declinePerMinute =
    totalDrop / elapsedMinutes;
```

The system can then estimate the time remaining before the reading reaches the warning threshold:

```
minutesUntilThreshold =
    (last.Value - LowReadingThreshold)
    / declinePerMinute;
```

This estimate is included in the recommendation where applicable.

Importantly, the system communicates that this is a projection based on the current rate of decline, rather than a guaranteed prediction.

2.5 Context-Aware Recommended Actions

The improved declining-trend logic also assigns different recommended actions depending on how quickly the sensor reading is decreasing.

The implementation uses three levels of decline:

Decline rate	Recommended response
At least 1.0 unit per minute	Rapid decline detected; inspect the sensor and connected equipment promptly.
At least 0.2 units per minute	Monitor the sensor closely and investigate if the decline continues.
Below 0.2 units per minute	Continue monitoring and investigate if the trend persists.

This makes the recommendation system more useful because the response is based on the rate of deterioration rather than simply identifying that a reading has decreased.

Result: Recommendations provide more context about the behaviour of a sensor and communicate actions that are more appropriate to the severity of the detected trend.

2.6 Duplicate Recommendation Prevention

Another improvement addressed repeated recommendations being generated for the same device.

The threshold recommendation logic now checks whether a recent recommendation already exists for the same device and severity.

If a matching recommendation exists within the five-minute update window, the system updates the existing record rather than creating another one.

An example of this approach is:

```
if (existing != null)
{
    existing.Timestamp = timestamp;
    existing.SensorValue = sensorValue;
    existing.DeploymentLocation = location;
    existing.Recommendation = recommendation;
    existing.RecommendedAction = action;
```

```
    return;
}
```

A similar approach was implemented for declining-trend recommendations.

This allows recommendations to remain current as new readings arrive without continuously creating duplicate records.

Result: The recommendation history becomes more manageable and less cluttered, while existing recommendations can reflect the latest sensor information.

2.7 Improved Actuator Command-Pattern Analysis

The actuator recommendation logic was also improved.

Previously, the recent command history could include commands from multiple actuators when evaluating repeated ON/OFF activity.

The updated implementation filters commands using the device MAC address:

```
List<CommandRecord> recentCommands =
    _commandHistory
        .Where(item =>
            item.DeviceMacAddress.Equals(
                macAddress,
                StringComparison.OrdinalIgnoreCase)
            &&
            (item.Command == "ON" ||
             item.Command == "OFF")
            &&
            item.RecordedAtUtc <= now
            &&
            now - item.RecordedAtUtc
                <= TimeSpan.FromMinutes(30))
        .ToList();
```

This ensures that command-pattern analysis is performed separately for each actuator.

Commands from unrelated devices can therefore no longer be combined to satisfy the repetition threshold for a recommendation.

Result: Actuator behaviour is evaluated independently, improving the accuracy and relevance of command-pattern recommendations.

3. WPF Architecture Refactoring — Persistent Shell Navigation

3.1 Feedback Identified

A significant usability and architectural issue existed within the original WPF navigation structure.

The application initially treated the telemetry dashboard and command dashboard as separate windows.

When users navigated between these dashboards, the application opened a new window and closed the previous one.

This created problems with maintaining the state of individual dashboard views.

In particular, sensor selections made through dropdown controls could be lost when navigating away from a dashboard and returning to it.

This meant users could be required to select their sensors repeatedly, disrupting the workflow.

The feedback therefore required a more consistent navigation structure that would allow the application to maintain dashboard instances and preserve relevant state.

3.2 Original Window-Based Navigation

The original implementation created a new dashboard window whenever a navigation button was selected.

For example, the previous telemetry navigation followed this pattern:

```
SensorDashboardWindow dashboard =
    new SensorDashboardWindow();
```

```
dashboard.Show();

this.Close();
```

The command dashboard followed a similar approach.

This meant that navigation was based on creating and closing windows instead of displaying different views within a persistent application container.

Consequently, navigating between dashboards could result in the previous dashboard instance being discarded.

This was particularly problematic for the sensor selection controls because their state belonged to the dashboard instance.

3.3 Introduction of a Persistent Shell

The application was refactored so that `MainWindow` became the central Shell responsible for hosting and navigating between dashboard views.

Instead of creating new windows during every navigation operation, the application now creates the dashboard instances once within the `MainWindow` constructor:

```
private readonly SensorDashboardWindow _telemetryView;
private readonly CommandDashboardWindow _commandView;

public MainWindow()
{
    InitializeComponent();

    _telemetryView = new SensorDashboardWindow();
    _commandView = new CommandDashboardWindow();

    ShowTelemetryView();
}
```

The dashboard views are retained as fields of the main window.

This is important because the same dashboard instances can be displayed again when the user navigates between sections.

The Shell therefore provides a persistent container for the application's main views.

3.4 ContentControl-Based Navigation

The main application window was updated to include a `ContentControl` named `MainContent`.

```
<ContentControl x:Name="MainContent"/>
```

This control acts as the hosting area for the different dashboard views.

The navigation methods were changed to display the existing dashboard instances inside this control.

For example:

```
private void ShowTelemetryView()
{
    MainContent.Content = _telemetryView;
}
```

The command dashboard uses the same approach:

```
private void ShowCommandView()
{
    MainContent.Content = _commandView;
}
```

The navigation buttons now call these methods rather than opening new windows.

```
private void BtnIngestion_Click(
    object sender,
    RoutedEventArgs e)
{
    ShowTelemetryView();
}

private void BtnCommandDashboard_Click(
    object sender,
    RoutedEventArgs e)
{
```

```
    ShowCommandView();
}
```

This creates a more consistent navigation experience because the Shell remains open while the displayed content changes.

3.5 Converting Dashboard Windows into UserControls

For the Shell architecture to work, the individual dashboards were converted from standalone WPF `Window` elements into reusable `UserControl` components.

For example, the telemetry dashboard changed from:

```
<Window
    x:Class="SmartXWPF.SensorDashboardWindow">
```

to:

```
<UserControl
    x:Class="SmartXWPF.SensorDashboardWindow">
```

The command dashboard underwent the same conversion.

Their code-behind declarations were also changed accordingly.

Previously:

```
public partial class SensorDashboardWindow : Window
```

After refactoring:

```
public partial class SensorDashboardWindow : UserControl
```

The same conversion was applied to `CommandDashboardWindow`.

This change allowed the dashboards to be hosted inside the main application window rather than operating as independent windows.

The individual dashboards could retain their own XAML layouts, controls and associated code-behind while becoming part of the central application interface.

3.6 Removing Independent Main Menu Navigation

The previous dashboards contained their own Main Menu buttons.

These buttons created a new `MainWindow`, displayed it and then closed the current dashboard.

For example, the previous navigation logic was:

```
MainWindow mainWindow = new MainWindow();

mainWindow.Show();

Close();
```

This logic was removed during the refactoring.

The navigation responsibility was moved to the persistent Shell, eliminating the need for individual dashboard windows to create new application windows when users wanted to return to the main menu.

Result: Navigation is now controlled centrally by `MainWindow`, reducing duplicated window-management logic and providing a more consistent application structure.

4. Sensor Selection Persistence and Dashboard State Management

4.1 Feedback Identified

The Shell refactoring addressed the broader navigation issue, but sensor selection also required specific handling when the sensor list was refreshed.

A selected sensor is represented by an object retrieved from the API. When the sensor collection is refreshed, the API can return new sensor object instances.

Simply retaining a reference to the previous object would therefore not be sufficient to guarantee that the selected sensor corresponds to an item in the refreshed collection.

The dashboard controller was improved to preserve the selected device using its stable MAC address.

4.2 Remembering the Selected Sensor

Within `DashboardController.RefreshSensorsAsync()`, the selected sensor's MAC address is stored before the refreshed sensor collection is loaded.

```
string? selectedMac =
    _state.SelectedSensor?.DeviceMacAddress;
```

The controller then retrieves the latest sensor profiles from the API and updates the dashboard:

```
var result = await _sensorApi.GetSensorsAsync();

if (!result.Success)
{
    _view.SetGatewayStatus(false);
    return;
}

_view.SetSensors(result.Sensors);
_view.SetGatewayStatus(true);
```

This ensures that the dashboard uses the latest sensor information.

4.3 Restoring the Selection

After refreshing the collection, the controller searches for the previously selected device using its MAC address.

```
SensorProfile? restoredSensor =
    result.Sensors.FirstOrDefault(sensor =>
        string.Equals(
            sensor.DeviceMacAddress,
            selectedMac,
            StringComparison.OrdinalIgnoreCase));
```

If the sensor is found, the controller restores it through the existing selection method:

```
if (restoredSensor != null)
{
    SelectSensor(restoredSensor);
}
else
{
    _state.SelectedSensor = null;
}
```

This approach is more reliable than depending on the previous sensor object reference because the MAC address acts as the stable identifier for the device.

If the selected device no longer exists in the refreshed API response, the controller clears the selection rather than retaining an invalid reference.

Result: Sensor selections can be restored after the sensor collection is refreshed, improving consistency and reducing unnecessary user interaction.

4.4 Preventing Unnecessary Dashboard Reinitialisation

The telemetry dashboard also received an initialization guard.

A Boolean field was introduced:

```
private bool _initialized;
```

The dashboard's Loaded event checks this value before initializing the controller:

```
private async void SensorDashboardWindow_Loaded(
    object sender,
    RoutedEventArgs e)
{
    if (_initialized)
        return;

    _initialized = true;

    await _controller.InitializeAsync();
}
```

This prevents the dashboard initialization routine from being executed repeatedly when the same view is loaded again.

It helps avoid unnecessary repeated loading operations when users navigate between views using the persistent Shell.

Result: The dashboard has more controlled initialization behaviour and avoids repeatedly executing its initial loading sequence.

4.5 Improving Input Preservation

The telemetry dashboard's category-control logic was also adjusted.

The previous implementation could overwrite the user's current sensor input when changing the selected sensor category.

The updated implementation separates the visibility of environmental/power input fields from actuator controls.

For example:

```
bool isActuator = selectedIndex == 2;

value.Visibility = isActuator
    ? Visibility.Collapsed
    : Visibility.Visible;

actuator.Visibility = isActuator
    ? Visibility.Visible
    : Visibility.Collapsed;
```

This ensures that the appropriate controls are displayed depending on the selected sensor category without unnecessarily replacing the user's current input with default values.

Result: The dashboard provides a more predictable interaction experience when users switch between different sensor categories.

5. Overall Impact of the Feedback Implementation

The Part 2 improvements addressed both functional and architectural concerns.

Area	Improvement implemented	Result
Input validation	Added checks for device identifiers, locations, readings and timestamps.	More reliable prediction input.
Reading history	Maintained chronological sensor readings and limited retained history.	More consistent historical analysis.
Trend detection	Added rate-of-decline calculations and estimated time to warning threshold.	More informative predictive recommendations.
Recommendation management	Updated recent matching recommendations instead of repeatedly creating duplicates.	Cleaner recommendation history.
Actuator analysis	Filtered command patterns by individual device.	More accurate actuator-specific analysis.
WPF navigation	Replaced independent dashboard windows with a persistent Shell.	More consistent navigation.
Dashboard architecture	Converted dashboard windows into reusable UserControls.	Dashboards can be hosted within the Shell.
Sensor selection	Restored selections using the device MAC address after refreshing sensors.	Reduced repeated sensor selection.
Dashboard initialization	Added an initialization guard.	Avoided unnecessary repeated initialization.

6. Conclusion

The feedback received during Part 2 identified areas where Smart-X could be improved beyond its initial functionality.

The predictive recommendation system was enhanced to perform more meaningful analysis of historical sensor readings, identify declining trends, estimate potential threshold breaches and provide more appropriate recommended actions.

The WPF application was also refactored to introduce a persistent Shell-based architecture. By converting the telemetry and command dashboards into UserControls and hosting them through a central ContentControl, the application moved away from repeatedly opening and closing independent windows.

Additional improvements to sensor selection persistence and dashboard initialization addressed the specific usability issues associated with navigating between features and refreshing sensor information.

Overall, these changes improved the reliability, usability and architectural consistency of Smart-X. More importantly, they demonstrate how the feedback received was translated into concrete implementation changes rather than being addressed through superficial interface adjustments.